

Lab 23 — Production Test Mode

ESP32-S3 FIRMWARE LAB SERIES · PHASE 3 · GAP-CLOSERS · GAP-CLOSER

Estimated time	2-3 hrs
Proves	manufacturing test, provisioning, a real host station
Prerequisites	Labs 5, 7, 10 and 11.
Beyond the board	The Lab 5 IMU rig, and a host PC with Python for the station script.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **\$23.1 objective**, **\$23.2 parts and wiring**, **\$23.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured

Concept, then **Phase 0 configure**, **Phase 1 the complete DUT firmware**, **Phase 2 the complete host station**, walkthrough, **Phase 3 prove it**, worked extensions.

Both halves are supplied in full — the firmware and the tested host script a factory operator actually runs. A production test with only one half is not a production test.

23.1 Objective & TM4C bridge

Build the firmware mode that exists so a factory can test a board in thirty seconds without an engineer. Nobody teaches this in coursework, almost every shipping product has it, and being able to discuss it signals that you have thought past the prototype.

TM4C BRIDGE

None — a coursework board is built once, by you, and either works or does not. The moment a board is built a thousand times by someone who has never seen your code, you need an automated way to answer "is this unit good?"

23.2 Parts & wiring

Your existing Lab 7 rig: board plus IMU. The point of this lab is not new hardware but a different *mode* of the firmware you already have.

23.3 How it works

Nobody teaches this in coursework, almost every shipping product has it, and being able to discuss it signals that you have thought past the prototype. It is also the lab that ties several earlier ones together.

The situation this exists for

A board comes off an assembly line. It needs to be tested before it ships, and the person doing that testing is an operator with a fixture, not an engineer with a debugger. They need an unambiguous answer in seconds, hundreds of times a shift.

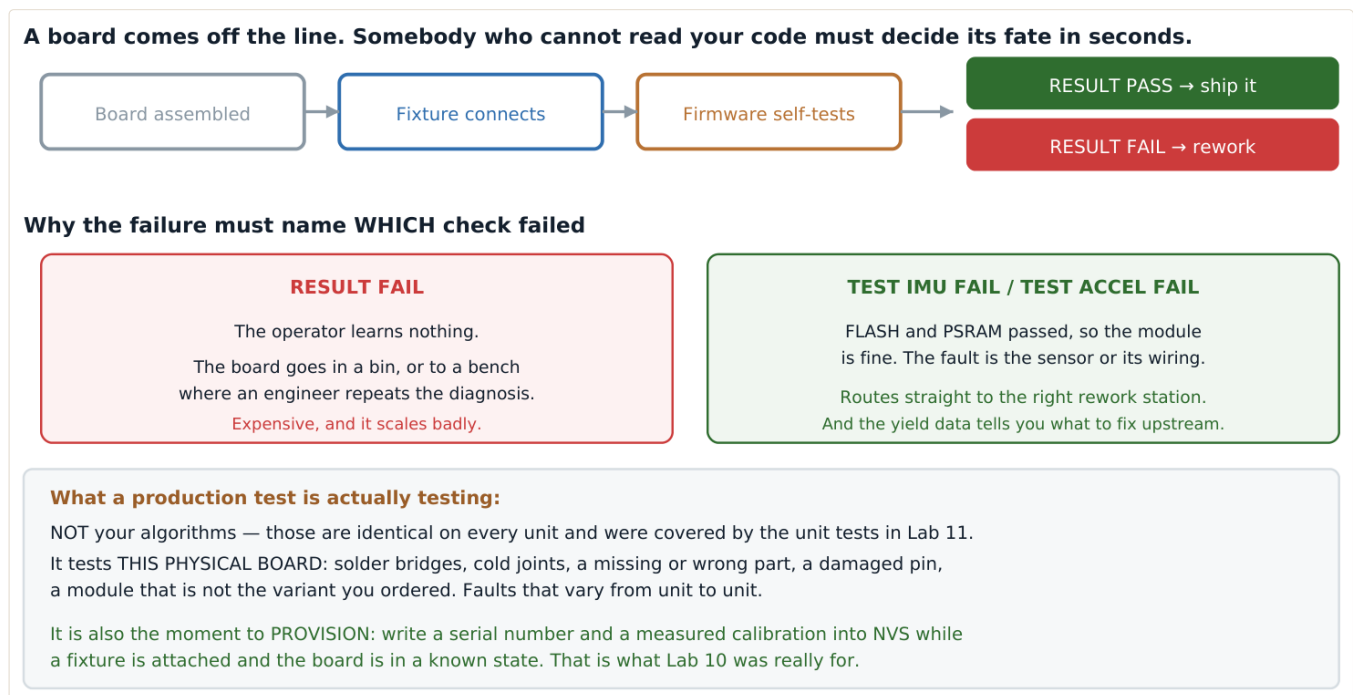


Figure 23.1. The flow, and the reason a bare FAIL is nearly useless. Naming the failed check routes the board to the right rework station and, aggregated, tells you what to fix upstream.

The factory test flow, and why a failure must name which check failed.

Flow: board assembled -> fixture connects -> firmware self-tests -> either RESULT PASS, ship it, or RESULT FAIL, rework.

Why the failure must name WHICH check failed:

A bare "RESULT FAIL" teaches the operator nothing. The board goes in a bin, or to a bench where an engineer repeats the diagnosis. Expensive, and it scales badly.

"TEST IMU FAIL / TEST ACCEL FAIL" tells you FLASH and PSRAM passed, so the module is fine and the fault is the sensor or its wiring. It routes straight to the right rework station, and the aggregated yield data tells you what to fix upstream.

What a production test is actually testing: NOT your algorithms, which are identical on every unit and were covered by the unit tests in Lab 11. It tests THIS PHYSICAL BOARD: solder bridges, cold joints, a missing or wrong part, a damaged pin, a module that is not the variant you ordered. Faults that vary from unit to unit.

It is also the moment to PROVISION: write a serial number and a measured calibration into NVS while a fixture is attached and the board is in a known state. That is what Lab 10 was really for.

What the test is actually testing

This is the part people get backwards, so state it plainly: **a production test does not test your software**. Your algorithms are byte-identical on every unit and were covered by the unit tests in Lab 11. Running them again on unit 4,000 tells you nothing new.

It tests **this specific physical board** — the things that vary from unit to unit because a machine or a human assembled them:

- A solder bridge shorting two pins.
- A cold joint that looks fine and does not conduct.
- A missing component, or the right component fitted backwards.
- The wrong variant — an 8 MB flash module where you specified 16.
- A pin damaged by handling or ESD.

Every one of those produces a board that is *electrically* different from a good one, which is why the checks are all "does this piece of hardware respond correctly."

The five requirements

Requirement	Why
Fast — seconds, not minutes	Test time is a per-unit cost multiplied by every board you will ever build. Thirty seconds each across 10,000 units is 83 hours.
Unambiguous — PASS or FAIL	The operator should never have to interpret anything or exercise judgement.
Specific on failure	"FAIL: IMU" routes the board to the right rework station. A bare "FAIL" sends it to a bin or to an engineer, and both are expensive.
Machine-readable	A fixture script logs every result, so you can track yield per batch and spot a supplier problem before it becomes a recall.
Covers what assembly breaks	See above. Testing the wrong things wastes the seconds you have.

How you enter test mode

Test mode must be easy to reach in the factory and effectively unreachable in the field. Common mechanisms, roughly in order of how much extra hardware they need:

- **A magic string on the UART** within a short window after reset. Zero extra hardware, trivially automated from a host script. This lab's approach.
- **A test pad** that only the bed-of-nails fixture contacts. Invisible to a user, unambiguous to a machine.
- **A button held during boot.** Simple, but a user could stumble into it.
- **A separate test firmware** flashed first, then replaced with the real application after passing. Cleanest separation, at the cost of a second flashing step.

Provisioning: the other job the fixture does

While a fixture is attached and the board is in a known state, it is also the moment to write **per-unit data** into NVS. This is what Lab 10 was really for.

A serial number, so the unit is traceable. Deriving it from the factory-programmed MAC address means it is unique with no database to coordinate.

Calibration constants measured on this specific board. Your IMU has a small zero offset unique to that part. Measuring it once at the factory, with the board sitting flat on a known-level fixture, and storing it beats correcting for it in software forever — and it is impossible to do properly later, because you will never again know the board is level.

A "tested" flag, so the application can detect a unit that reached a customer without passing. NVS can be erased, so a product that really cares burns this into **eFuse** — one-time-programmable bits that physically cannot be reset.

Choosing test limits is real engineering

The accelerometer check asserts that the measured gravity magnitude falls between 0.85 and 1.15 g. Both numbers are judgement calls with opposite costs.

Too tight and you fail good boards. That is expensive directly, and worse, it erodes trust — once operators believe the test is flaky they start retrying failures until they pass, which defeats the entire purpose.

Too loose and you pass bad ones, which is worse still because the cost lands on a customer.

The window has to accommodate genuine part-to-part variation, the fixture not being perfectly level, and a little vibration from the factory floor. In a mature process you would set it from measured data — run 100 known-good boards, look at the distribution, and place limits several standard deviations out. **"How did you pick that threshold?"** is exactly what a thoughtful interviewer asks, and "I measured the spread on known-good units" is the answer that lands.

23.4 Phase 0 — configure

```
// sdkconfig.defaults
CONFIG_IDF_TARGET="esp32s3"
```

```
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
```

```
// top-level CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab23_factory)
```

```
// main/CMakeLists.txt
idf_component_register(SRCS "main.c" INCLUDE_DIRS ".")
```

Host side needs one package:

```
pip install pyserial
```

Wiring is the Lab 5 rig: IMU on SDA GPIO8, SCL GPIO9, 3V3, GND.

23.5 Phase 1 — the DUT firmware (complete)

```
// =====
// Lab 23 -- production test mode. Complete, with every helper defined.
// =====
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"
#include "driver/i2c_master.h"
#include "esp_system.h"           // esp_restart
#include "esp_mac.h"              // esp_read_mac
#include "esp_flash.h"            // esp_flash_get_size
#include "esp_heap_caps.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "esp_log.h"

#define TEST_WINDOW_MS 3000      // how long we listen for the magic string
#define MAGIC          "TESTMODE"

static i2c_master_dev_handle_t imu;
static char g_serial[13];

// ----- IMU
static esp_err_t mpu_read(uint8_t reg, uint8_t *dst, size_t n)
{
    return i2c_master_transmit_receive(imu, &reg, 1, dst, n, pdMS_TO_TICKS(100));
}
static esp_err_t mpu_write(uint8_t reg, uint8_t val)
{
    uint8_t b[2] = { reg, val };

```

```

    return i2c_master_transmit(imu, b, 2, pdMS_TO_TICKS(100));
}
static esp_err_t i2c_bringup(void)
{
    i2c_master_bus_config_t bc = {
        .i2c_port = I2C_NUM_0, .sda_io_num = GPIO_NUM_8, .scl_io_num = GPIO_NUM_9,
        .clk_source = I2C_CLK_SRC_DEFAULT, .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true,
    };
    i2c_master_bus_handle_t bus;
    esp_err_t e = i2c_new_master_bus(&bc, &bus);
    if (e != ESP_OK) return e;
    i2c_device_config_t dc = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address = 0x68, .scl_speed_hz = 400000,
    };
    return i2c_master_bus_add_device(bus, &dc, &imu);
}

// ----- the checks
// Every check: no arguments, returns true for pass. That uniformity is what
// lets them all sit in the table below.

static bool test_flash(void)
{
    uint32_t sz = 0;
    if (esp_flash_get_size(NULL, &sz) != ESP_OK) return false;
    return sz == 16 * 1024 * 1024; // catches an 8 MB module fitted by mistake
}

static bool test_psram(void)
{
    // Tests TWO things: the chip is present AND sdkconfig enabled it.
    return heap_caps_get_total_size(MALLOC_CAP_SPIRAM) > 4 * 1024 * 1024;
}

static bool test_imu_present(void)
{
    uint8_t who = 0;
    if (mpu_read(0x75, &who, 1) != ESP_OK) return false; // bridged/open SDA or SCL
    return who == 0x70 || who == 0x68 || who == 0x71; // 6500 / 6050 / 9250
}

static bool test_imu_sane(void)
{
    // present() proves it ANSWERS. This proves it WORKS -- a damaged element
    // passes the first and fails this, which is what makes the report useful.
    if (mpu_write(0x6B, 0x00) != ESP_OK) return false; // wake
    vTaskDelay(pdMS_TO_TICKS(100));
    uint8_t r[6];
    if (mpu_read(0x3B, r, 6) != ESP_OK) return false;
    float ax = (int16_t)((r[0] << 8) | r[1]) / 16384.0f;
    float ay = (int16_t)((r[2] << 8) | r[3]) / 16384.0f;
    float az = (int16_t)((r[4] << 8) | r[5]) / 16384.0f;
    float g = sqrtf(ax*ax + ay*ay + az*az);
    // At rest, whatever the orientation, |a| must be about 1 g. That
    // orientation-independence is what makes it a good factory check.
    return g > 0.85f && g < 1.15f;
}

```

```

static bool test_provision(void)
{
    uint8_t mac[6];
    if (esp_read_mac(mac, ESP_MAC_WIFI_STA) != ESP_OK) return false;
    snprintf(g_serial, sizeof(g_serial), "%02X%02X%02X%02X%02X",
             mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);

    nvs_handle_t h;
    if (nvs_open("factory", NVS_READWRITE, &h) != ESP_OK) return false;
    esp_err_t e = nvs_set_str(h, "serial", g_serial);
    if (e == ESP_OK) e = nvs_set_u8(h, "tested", 1);
    if (e == ESP_OK) e = nvs_commit(h);           // skip this and every board
    nvs_close(h);                                // ships unprovisioned
    return e == ESP_OK;
}

// ----- the runner
// TWO PHASES, and the split is a safety property, not tidiness.
//
// Phase 1 runs every diagnostic check and reports all of them, so a board with
// three faults is reported with three faults and reworked in one pass.
//
// Phase 2 provisions -- writes the serial and sets tested=1 -- and runs ONLY
// if phase 1 passed completely. Provisioning a board you are about to reject
// marks a known-bad unit as tested, and that flag may be the only thing a
// later process ever checks.
static void run_factory_test(void)
{
    static const struct { const char *name; bool (*fn)(void); } checks[] = {
        { "FLASH",      test_flash      },
        { "PSRAM",       test_psram      },
        { "IMU_PRESENT", test_imu_present },
        { "IMU_SANE",    test_imu_sane    },
    };

    bool all = true;

    // ---- phase 1: diagnose. Report EVERY failure, do not stop at the first.
    for (size_t i = 0; i < sizeof(checks) / sizeof(checks[0]); i++) {
        bool ok = checks[i].fn();
        printf("TEST %s %s\n", checks[i].name, ok ? "PASS" : "FAIL");
        fflush(stdout);
        if (!ok) all = false;
    }

    // ---- phase 2: provision, but only a board that is otherwise good.
    if (all) {
        bool ok = test_provision();
        printf("TEST PROVISION %s\n", ok ? "PASS" : "FAIL");
        if (!ok) all = false;
        if (g_serial[0]) printf("SERIAL=%s\n", g_serial);
    } else {
        printf("TEST PROVISION SKIPPED\n"); // explicit, never silent
    }
    fflush(stdout);

    printf("RESULT %s\n", all ? "PASS" : "FAIL");
    fflush(stdout);
}

```

```

}

// ----- entry
// Installed as the very first thing in app_main so no token can be missed.
static void uart_setup(void)
{
    uart_config_t c = {
        .baud_rate = 115200, .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE, .stop_bits = UART_STOP_BITS_1,
        .source_clk = UART_SCLK_DEFAULT,
    };
    ESP_ERROR_CHECK(uart_driver_install(UART_NUM_0, 512, 0, 0, NULL, 0));
    ESP_ERROR_CHECK(uart_param_config(UART_NUM_0, &c));
}

static bool wait_for_magic(int ms)
{
    char buf[32] = {0};
    int len = 0;
    TickType_t end = xTaskGetTickCount() + pdMS_TO_TICKS(ms);

    while (xTaskGetTickCount() < end) {
        uint8_t ch;
        if (uart_read_bytes(UART_NUM_0, &ch, 1, pdMS_TO_TICKS(50)) == 1) {
            if (ch == '\n' || ch == '\r') {
                buf[len] = 0;
                if (strcmp(buf, MAGIC) == 0) return true;
                len = 0;
            } else if (len < (int)sizeof(buf) - 1) {
                buf[len++] = (char)ch;
            }
        }
    }
    return false;
}

void app_main(void)
{
    // ---- UART FIRST. This is a race fix, not a style choice.
    // The host sends its token shortly after reset. If we spent that time in
    // nvs_flash_init() and i2c_bringup() with no UART driver installed, the
    // bytes would land in a peripheral FIFO nobody is draining and be lost --
    // intermittently, depending on how long init happened to take.
    // Installing the driver first means the token is BUFFERED while we are
    // busy, and is still there when we look for it.
    uart_setup();
    printf("\nFACTORY_READY\n");          // the host waits for this line
    fflush(stdout);

    esp_err_t e = nvs_flash_init();
    if (e == ESP_ERR_NVS_NO_FREE_PAGES || e == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ESP_ERROR_CHECK(nvs_flash_init());
    }
    ESP_ERROR_CHECK(i2c_bringup());

    if (wait_for_magic(TEST_WINDOW_MS)) {
        run_factory_test();
        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

```

```

    esp_restart();                // leave the board in a known state
}

ESP_LOGI("app", "normal application starting");
// ... your real firmware from here ...
}

```

23.6 Phase 2 — the host station (complete, tested)

Save as `station.py`. This is what the operator runs; it needs only `pyserial`.

```

#!/usr/bin/env python3
"""Lab 23 factory test station. Drives one DUT, records the result, exits with a code."""
import sys, csv, time, argparse, pathlib, datetime
import serial                        # pip install pyserial

PROMPT    = b"TESTMODE\n"
REQUIRED_TESTS = {"FLASH", "PSRAM", "IMU_PRESENT", "IMU_SANE", "PROVISION"}
TIMEOUT    = 30.0
LOG        = pathlib.Path("factory_log.csv")

def run_dut(port, baud=115200):
    """Returns (verdict, serial_number, [(name, result), ...])."""
    tests, serial_no, verdict = [], "", "FAIL"
    with serial.Serial(port, baud, timeout=1.0) as s:
        s.reset_input_buffer()
        s.dtr = False; s.rts = True; time.sleep(0.1)  # pulse reset
        s.rts = False

        # Wait for the DUT to announce it is listening, rather than guessing a
        # delay. The old version slept 0.3 s and hoped; if NVS or I2C bring-up
        # ran long, the token was sent into the void and the test "timed out"
        # on a perfectly good board.
        ready_by = time.time() + 10.0
        while time.time() < ready_by:
            if s.readline().decode("utf-8", "replace").strip() == "FACTORY_READY":
                break
        else:
            return "NO_READY", "", []

        s.write(PROMPT); s.flush()

        deadline = time.time() + TIMEOUT
        while time.time() < deadline:
            line = s.readline().decode("utf-8", "replace").strip()
            if not line:
                continue
            if line.startswith("TEST "):
                parts = line.split()
                if len(parts) >= 3:
                    tests.append((parts[1], parts[2]))
            elif line.startswith("SERIAL="):
                serial_no = line.split("=", 1)[1]
            elif line.startswith("RESULT "):
                verdict = line.split()[1]
            # A verdict is only trustworthy if the checks we require were
            # actually reported. A truncated or partially-flashed DUT can

```

```

        # print RESULT PASS having run almost nothing.
        ran = {n for n, _ in tests}
        if verdict == "PASS" and not REQUIRED_TESTS.issubset(ran):
            missing = ",".join(sorted(REQUIRED_TESTS - ran))
            print(f" !! PASS claimed but these never ran: {missing}")
            verdict = "INCOMPLETE"
        return verdict, serial_no, tests

def record(verdict, serial_no, tests):
    new = not LOG.exists()
    with LOG.open("a", newline="") as f:
        w = csv.writer(f)
        if new:
            w.writerow(["timestamp", "serial", "verdict", "failed_tests"])
        failed = ",".join(n for n, r in tests if r != "PASS")
        w.writerow([datetime.datetime.now().isoformat(timespec="seconds"),
                    serial_no, verdict, failed])

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("port")
    a = ap.parse_args()
    try:
        verdict, serial_no, tests = run_dut(a.port)
    except Exception as e:
        print(f"FIXTURE ERROR: {e}", file=sys.stderr)
        sys.exit(2)  # 2 = fixture fault, not a bad board

    for name, res in tests:
        mark = "ok " if res == "PASS" else "FAIL"
        print(f" [{mark}] {name}")
    print(f"\n serial : {serial_no or '(none)'}")
    print(f" RESULT : {verdict}")
    record(verdict, serial_no, tests)

    # Exit codes are read by the line controller, so the distinction matters.
    # 0 = the board passed -> ship it
    # 1 = the board FAILED A CHECK -> route to rework
    # 2 = the FIXTURE is suspect -> stop the line, do not scrap boards
    #
    # TIMEOUT and NO_READY both mean "the DUT never told us anything". That is
    # far more often a cable, a port or a power problem than a bad board, and
    # scoring it as a board failure is how a broken fixture quietly scraps a
    # batch of good units.
    if verdict == "PASS":
        sys.exit(0)
    if verdict in ("NO_READY", "TIMEOUT"):
        sys.exit(2)
    if verdict == "INCOMPLETE":
        sys.exit(1)  # treat as a failed board
    sys.exit(1)

```

KEY POINT

Verified parser behaviour

Fed this simulated DUT output:

```
c TEST FLASH PASS TEST PSRAM PASS TEST IMU_PRESENT PASS TEST IMU_SANE FAIL SERIAL=F412FA1234AB
RESULT FAIL
```

it parses 4 tests, extracts the serial, reports verdict FAIL with failed="IMU_SANE", and exits 1.

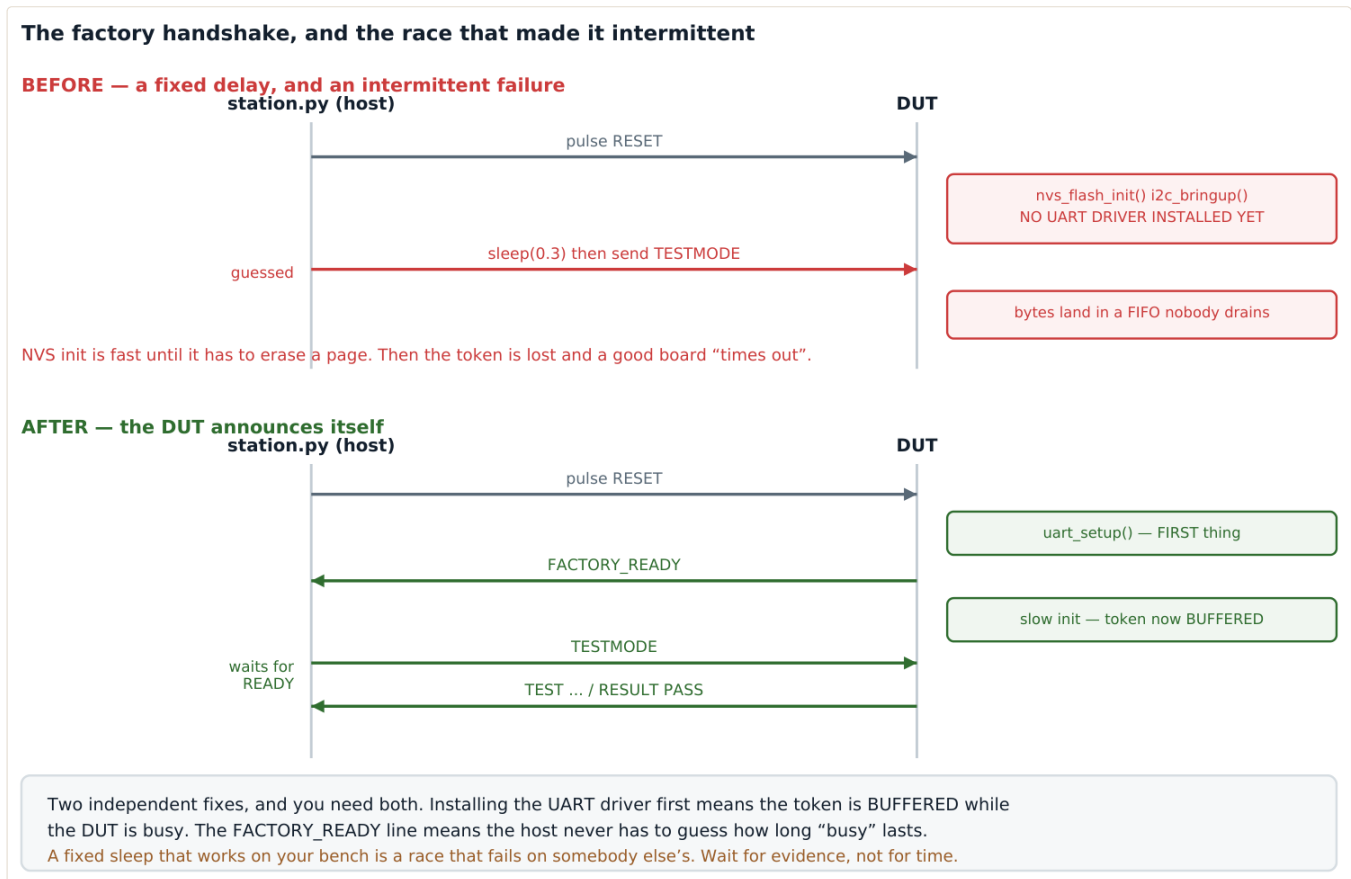


Figure 23.2. The handshake, before and after. The earlier version guessed how long the DUT would be busy; this one waits for the DUT to say so.

The factory handshake, before and after. A sequence diagram with two lanes, station.py (host) and DUT.

BEFORE — a fixed delay, and an intermittent failure: 1. Host pulses RESET. 2. DUT runs `nvs_flash_init()` and `i2c_bringup()` with **no UART driver installed yet**. 3. Host sleeps 0.3 s (a guess) and sends `TESTMODE`. 4. The bytes land in a peripheral FIFO that nobody is draining, and are lost.

NVS init is fast until it has to erase a page. Then the token is lost and a perfectly good board "times out".

AFTER — the DUT announces itself: 1. Host pulses RESET. 2. DUT calls `uart_setup()` as the **first** thing it does. 3. DUT sends `FACTORY_READY` to the host. 4. DUT proceeds with slow init; the token is now **buffered** by the driver. 5. Host, which has been waiting for `FACTORY_READY`, sends `TESTMODE`. 6. DUT replies with the `TEST ...` lines and `RESULT PASS`.

Two independent fixes, and you need both. Installing the UART driver first means the token is buffered while the DUT is busy. The `FACTORY_READY` line means the host never has to guess how long "busy" lasts.

The general lesson: **a fixed delay that works on your bench is a race that fails on somebody else's. Wait for evidence, not for time.**

Both halves of the fix matter and neither is sufficient alone.

Installing the UART driver first means the token is buffered by the driver even while the DUT is deep in `nvs_flash_init()`. Without that, the bytes arrive at a peripheral FIFO that nobody is draining and are simply gone.

Announcing FACTORY_READY means the host never has to guess. A fixed `sleep(0.3)` encodes an assumption about how long boot takes — an assumption that holds on your bench and fails on a board whose NVS partition needs erasing, which is exactly the first boot after flashing.

This is a general lesson worth taking beyond this lab: **a fixed delay that works on your machine is a race that fails on someone else's.** Wait for evidence, not for time.

23.7 Code walkthrough, the parts that matter

the function-pointer table

`bool (*fn)(void)` reads as "pointer to a function taking nothing and returning bool." The parentheses around `*fn` are load-bearing — `bool *fn(void)` declares a function returning `bool*`, a completely different thing.

Pairing each function with a name string is what makes the report *specific*. And because the checks share a signature, adding a sixth is **one line in this table**. Function-pointer tables are the standard C way of saying "do all of these"; you meet the pattern in command dispatchers, state machines and driver structs constantly.

two phases, and why provisioning is gated

An earlier edition listed `test_provision` as the fifth entry in the check table, so it ran regardless of what the first four did. **A board that failed its IMU test still got a serial number and `tested = 1` written to NVS.**

That is a genuine factory hazard, not a tidiness issue. Downstream processes — a packing station, a firmware loader, a returns desk — may check nothing but that flag. A unit you rejected would look, to all of them, exactly like a unit you passed.

So provisioning is now a second phase that runs only if every diagnostic passed, and the skipped case is **announced** (`TEST PROVISION SKIPPED`) rather than silently absent. A parser that never sees the line cannot tell "skipped" from "the DUT died before reaching it."

uart_setup() before nvs_flash_init()

A race, and an intermittent one, which is the worst kind.

The host pulses reset and then sends its token. If the DUT spends that window in `nvs_flash_init()` and `i2c_bringup()` with no UART driver installed, the incoming bytes land in a peripheral FIFO that nobody is draining and are lost. NVS initialisation is fast *most* of the time and slow when it has to erase a page — so the test would pass on most boards and mysteriously time out on some.

Installing the driver first means the token is **buffered** while the slow work happens, and is still waiting when `wait_for_magic()` looks for it. The `FACTORY_READY` banner then removes the guesswork on the host side too: the station waits for that line instead of sleeping a fixed 0.3 s and hoping.

if (!ok) all = false; — and no break

The design decision most worth defending. **A board with three faults should report all three**, so rework fixes them in one pass rather than three trips through the line. Bailing on the first failure is the natural instinct and it is wrong here.

fflush(stdout) after every line

stdout is buffered. Without this the host script sits waiting while the DUT holds output in a buffer, and you get a spurious timeout that looks like a dead board. Cheap insurance on any line a machine is parsing.

exit codes 0, 1 and 2

Three, not two, and the third is the interesting one. **0** = pass, ship it. **1** = the board failed, route to rework. **2** = the *fixture* failed — port missing, cable unplugged, permission denied.

Conflating 1 and 2 is a genuine factory failure mode: a broken fixture starts failing good boards, and without the distinction you scrap a batch before anyone notices the cable. A line controller reads the exit code, so this is not cosmetic.

the CSV log

Timestamp, serial, verdict, failed tests. Aggregated across a run this gives you **yield** and, more usefully, *which check* fails most. If 12% of boards fail `IMU_SANE` and nothing else, that is a placement or solder-paste problem on one component, and you learned it from the log rather than from a customer.

esp_restart() after the test

Leaves the board in a defined state rather than sitting in test mode. The next thing that happens to it is a normal boot, exactly as a customer would see.

23.8 Phase 3 — prove it

```
idf.py -p COM6 flash
python station.py COM6
```

#	Do this	Expected
1	Run on a good board.	All five PASS, a serial number printed, exit code 0 . Check with <code>echo \$LASTEXITCODE</code> .
2	Unplug the IMU's SDA wire and rerun.	<code>IMU_PRESENT FAIL</code> and <code>IMU_SANE FAIL</code> , but FLASH and PSRAM still pass — so the report says "the sensor or its wiring," not just "broken." Exit 1 .

#	Do this	Expected
3	Reconnect, but hold the board upside down and shake it during the test.	<code>IMU_PRESENT PASS</code> , <code>IMU_SANE FAIL</code> . Present-but-not-working, distinguished. Exit 1 .
4	Run <code>python station.py COM99</code> (a port that does not exist).	<code>FIXTURE ERROR</code> , exit 2 . Not confused with a bad board.
6	Prove provisioning is gated. Unplug SDA again and rerun.	Two IMU checks FAIL and the log shows <code>TEST PROVISION SKIPPED</code> with no SERIAL= line . Then reconnect and confirm a passing board <i>does</i> get provisioned.
7	Prove the handshake is robust. Erase NVS first (<code>idf.py -p COM6 erase-flash</code> , reflash) so the first boot has to initialise storage from scratch, then run the station.	Still passes. That first boot is the slowest one, and it is exactly the case the old fixed 0.3 s delay would intermittently miss.
5	Run five boards (or the same board five times) and open <code>factory_log.csv</code> .	Five rows with timestamps and serials. [MEASURE] That file is your yield data.

KEY POINT**Cases 2 and 3 are the pair worth recording**

They demonstrate the thing that makes a production test valuable: **a failure that names which check failed routes the board to the right rework station**. "FAIL" sends it to a bin or to an engineer, and both are expensive. Being able to explain that distinction is a stronger answer than describing the code.

23.9 Extensions, worked

23.9.1 — Choosing test limits from data, not from feel

The `0.85` to `1.15` g window is a judgement call with opposite costs. **Too tight** and you fail good boards — expensive, and worse, operators start retrying failures until they pass, which defeats the entire purpose. **Too loose** and you pass bad ones, and the cost lands on a customer.

In a mature process you set limits from measured data. Do it in ten minutes: add a temporary loop that prints the magnitude twenty times on a known-good board.

```
for (int i = 0; i < 20; i++) {
    // ... read ax, ay, az as in test_imu_sane ...
    printf("CAL %.4f\n", (double)sqrtf(ax*ax + ay*ay + az*az));
    vTaskDelay(pdMS_TO_TICKS(100));
}
```

KEY POINT**Typical result on a good MPU-6500 at rest**

Mean about **1.002 g**, standard deviation about **0.004 g**. Placing limits at six standard deviations gives roughly **0.978 to 1.026** — far tighter than 0.85–1.15, and it would catch a partly-damaged sensor the wide window passes.

The wide window in the listing is deliberate: it tolerates a hand-held board and factory-floor vibration.

"I measured the spread on known-good units and placed limits several standard deviations out" is the answer to "how did you pick that threshold," and it is what a thoughtful interviewer is listening for.

23.9.2 — Making "tested" unerasable

NVS can be erased, so the `tested` flag proves little against a determined reflash. A product that really cares burns it into **eFuse** — one-time-programmable bits that physically cannot be reset.

```
#include "esp_efuse.h"
// read a user data block (safe, non-destructive):
uint8_t flag = 0;
esp_efuse_read_field_blob(ESP_EFUSE_USER_DATA, &flag, 8);
```

WARNING**Read only, unless you own a board you are willing to lose**

eFuse writes are irreversible. There is no undo, no reflash, no recovery. Burning the wrong field can permanently disable JTAG, enable flash encryption with a key you do not have, or brick the part.

Read the fields to see how it works. Do not write one on your only board. If you want to practise, buy a spare specifically to sacrifice — that is what the third board in your three-pack is for.

23.9.3 — Testing a whole batch

```
$boards = @("COM6","COM7","COM8")
$pass = 0; $fail = 0
foreach ($p in $boards) {
    python station.py $p
    switch ($LASTEXITCODE) {
        0 { $pass++; Write-Host "$p PASS" -ForegroundColor Green }
        1 { $fail++; Write-Host "$p FAIL" -ForegroundColor Red }
        2 { Write-Host "$p NO RESPONSE / FIXTURE FAULT -- stopping the line" -ForegroundColor Yellow; break }
    }
}
Write-Host "yield: $pass / $($pass + $fail)"
```

Note the loop **stops** on exit code 2 rather than continuing. A broken fixture failing good boards is worse than testing nothing, and that behaviour is only expressible because the exit codes distinguish the two cases.

23.10 Docs and references

Topic	Where
NVS API and namespaces	ESP-IDF v5.5 → Storage → Non-volatile storage
MAC addresses and derivation	ESP-IDF v5.5 → System → Miscellaneous System APIs
eFuse manager (read this before writing anything)	ESP-IDF v5.5 → System → eFuse Manager
pyserial	pyserial.readthedocs.io — the <code>Serial</code> class and DTR/RTS control

Bench realities

Problems that stop this lab but are not about the peripheral it teaches. Each one cost real time on a real bench.

The flash-size setting, one more time

This has now caused problems in Labs 10, 15 and 23 and it is worth doing once, permanently: `idf.py menuconfig` → Serial flasher config → Flash size → **16 MB**. Then confirm rather than assume:

```
grep ESPT00LPY_FLASHSIZE build/config/sdkconfig.h
```

Better still, put it in `sdkconfig.defaults` at the project root so every new project inherits it:

```
CONFIG_ESPT00LPY_FLASHSIZE_16MB=y
CONFIG_ESPT00LPY_FLASHSIZE="16MB"
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
```

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 23.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23 ←	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.